

Work_Trial_Task

July 31, 2025

First, I import the data:

```
[1]: import pandas as pd
import glob
import os

def load_and_index_csvs(dir_paths):
    # Load CSVs
    dfs = []
    for dir_path in dir_paths:
        expanded_dir = os.path.expanduser(dir_path)
        pattern = os.path.join(expanded_dir, '**', '*.csv')
        for filepath in glob.glob(pattern, recursive=True):
            df = pd.read_csv(filepath)
            dfs.append(df)

    if not dfs:
        return pd.DataFrame()

    combined = pd.concat(dfs, ignore_index=True)

    # Convert ts_event nanoseconds to datetime
    combined['ts_event_dt'] = pd.to_datetime(combined['ts_event'], unit='ns')
    # Extract date for indexing
    combined['date'] = pd.to_datetime(combined['ts_event'], unit='ns').dt.date

    # Ensure overall chronological order
    combined.sort_values(['symbol', 'ts_event'], inplace=True)

    # Set MultiIndex: first level symbol, second level event_date
    indexed = combined.set_index(['symbol', 'date'])
    # Optionally sort the MultiIndex
    indexed = indexed.sort_index()

    return indexed

paths = ['~/Downloads/Quant/SOUN', '~/Downloads/Quant/FROG', '~/Downloads/Quant/
↳ CRWV']
```

```
df_indexed = load_and_index_csvs(paths)
```

Next, for every minute long interval from market open to close, I average the first non-zero ask-size depth across all quote events for each individual symbol-day panel, then take the cross-sectional mean of those minute-level values over all symbols and days. Then I plot the 390-point series:

```
[2]: import matplotlib.pyplot as plt

# Reset the index to get symbol / ts_event_dt back as columns
df = df_indexed.reset_index()

# Make ts_event_dt timezone-aware ET in all cases
if df['ts_event_dt'].dt.tz is None:
    df['ts_event_dt'] = (
        df['ts_event_dt']
        .dt.tz_localize('UTC')          # assume current values are UTC
        .dt.tz_convert('America/New_York')
    )
else:
    df['ts_event_dt'] = df['ts_event_dt'].dt.tz_convert('America/New_York')

# Derive plain-date column if missing
if 'date' not in df.columns:
    df['date'] = df['ts_event_dt'].dt.date

# Floor to nearest minute
df['minute_bin'] = df['ts_event_dt'].dt.floor('min')

# First non-zero ask_ct_N per row
ask_ct_cols = [f'ask_sz_{i:02d}' for i in range(10)]
def first_nonzero_depth(row):
    for col in ask_ct_cols:
        if col in row and row[col] > 0:
            return row[col]
    return 0
df['depth'] = df.apply(first_nonzero_depth, axis=1)

# Compute minute averages of Dt
minute_dt = (
    df.groupby(['symbol', 'date', 'minute_bin'])['depth']
    .mean()
    .reset_index(name='Dt')
)

# Minutes since 09:30 Eastern
minute_dt['minutes_since_open'] = (
    minute_dt['minute_bin'].dt.hour * 60
```

```

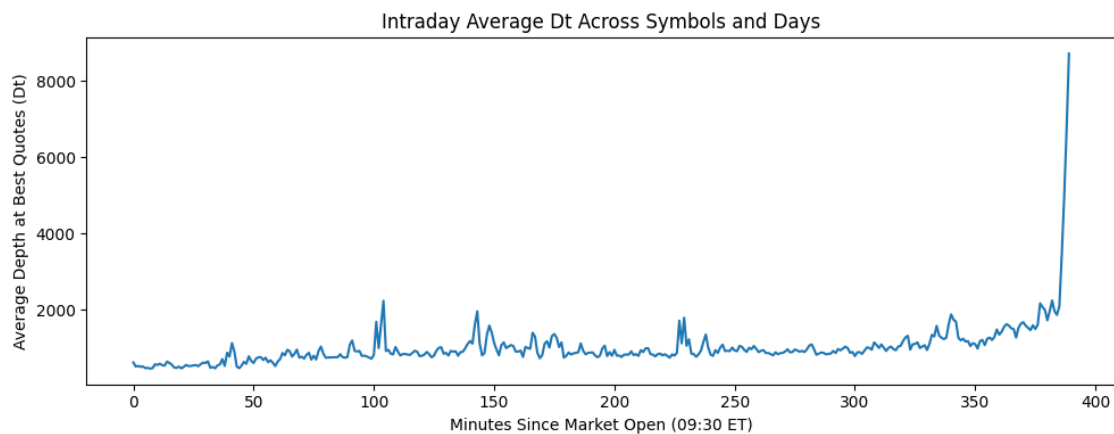
+ minute_dt['minute_bin'].dt.minute
- (9 * 60 + 30)
)

# Keep regular session minutes
trading_dt = minute_dt[
    (minute_dt['minutes_since_open'] >= 0) &
    (minute_dt['minutes_since_open'] < 390)
]

# Average across symbols and days
avg_dt = (
    trading_dt.groupby('minutes_since_open')['Dt']
    .mean()
    .reset_index()
)

# Plot
plt.figure(figsize=(10, 4))
plt.plot(avg_dt['minutes_since_open'], avg_dt['Dt'])
plt.xlabel('Minutes Since Market Open (09:30 ET)')
plt.ylabel('Average Depth at Best Quotes (Dt)')
plt.title('Intraday Average Dt Across Symbols and Days')
plt.tight_layout()
plt.show()

```



Next, I fit a penalised cubic B-spline (P-spline) with six interval knots to the minute-level intraday average of Dt, choosing a smoothness penalty by generalized cross-validation. I then plot the fit against the raw minute averages.

```

[ ]: import numpy as np
      from patsy import dmatrix

```

```

import statsmodels.api as sm

# Minute grid and target
x = avg_dt['minutes_since_open'].to_numpy()
y = avg_dt['Dt'].to_numpy()

# Low-rank cubic B-spline basis
knots = np.linspace(60, 330, 6)
Bx = dmatrix("bs(x, knots=knots, degree=3, include_intercept=True)",
             {"x": x}, return_type='dataframe').to_numpy()

# Penalty matrix: second-order differences of coefficients
D = np.diff(np.eye(Bx.shape[1]), n=2, axis=0)
pen = 2

# Generalised cross-validation to pick lambda (ridge-style)
lams = 10.0 ** np.linspace(-4, 4, 100)
gcv = []
for lam in lams:
    XtX = Bx.T @ Bx + lam * (D.T @ D)
    coef = np.linalg.solve(XtX, Bx.T @ y)
    hat = Bx @ np.linalg.solve(XtX, Bx.T @ y)
    rss = np.sum((y - hat) ** 2)
    df1 = np.trace(hat)
    gcv.append(rss / (len(y) - df1) ** 2)
lam_opt = lams[np.argmin(gcv)]

# Re-fit with optimal lambda
XtX = Bx.T @ Bx + lam_opt * (D.T @ D)
coef = np.linalg.solve(XtX, Bx.T @ y)

# Callable spline, its derivative, and an antiderivative
def _basis(t):
    return dmatrix("bs(t, knots=knots, degree=3, include_intercept=True)",
                  {"t": np.asarray(t)}, return_type='dataframe').to_numpy()

def Dt_hat(t):
    return _basis(t) @ coef

# First derivative via analytic derivative of B-spline basis
def _dbasis(t):
    return dmatrix("bs(t, knots=knots, degree=3, include_intercept=True,
    ↪deriv=1)",
                  {"t": np.asarray(t)}, return_type='dataframe').to_numpy()

def dDt_dt(t):
    return _dbasis(t) @ coef

```

```

# Quick check: smooth curve vs. minute points
dense_x = np.linspace(0, 389, 390 * 60)
plt.figure(figsize=(10, 4))
plt.plot(dense_x, Dt_hat(dense_x), label='P-spline')
plt.scatter(x, y, s=8, alpha=0.4, label='Minute averages')
plt.xlabel('Minutes Since Market Open (09:30 ET)')
plt.ylabel('Average Depth at Best Quotes (Dt)')
plt.title('Intraday Liquidity Curve - Penalised B-spline')
plt.legend(); plt.tight_layout(); plt.show()

```

```

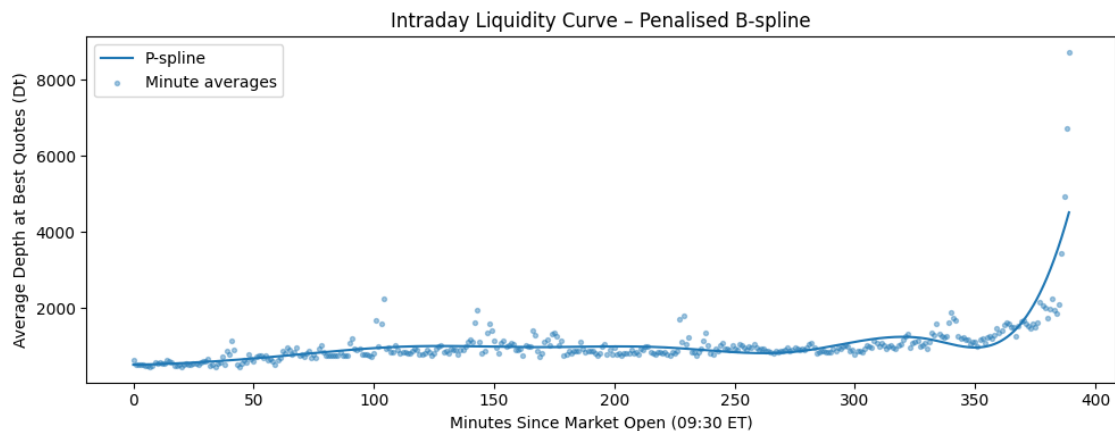
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:2
1: RuntimeWarning: divide by zero encountered in matmul
    XtX = Bx.T @ Bx + lam * (D.T @ D)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:2
1: RuntimeWarning: overflow encountered in matmul
    XtX = Bx.T @ Bx + lam * (D.T @ D)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:2
1: RuntimeWarning: invalid value encountered in matmul
    XtX = Bx.T @ Bx + lam * (D.T @ D)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:2
2: RuntimeWarning: divide by zero encountered in matmul
    coef = np.linalg.solve(XtX, Bx.T @ y)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:2
2: RuntimeWarning: overflow encountered in matmul
    coef = np.linalg.solve(XtX, Bx.T @ y)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:2
2: RuntimeWarning: invalid value encountered in matmul
    coef = np.linalg.solve(XtX, Bx.T @ y)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:2
3: RuntimeWarning: divide by zero encountered in matmul
    hat = Bx @ np.linalg.solve(XtX, Bx.T)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:2
3: RuntimeWarning: overflow encountered in matmul
    hat = Bx @ np.linalg.solve(XtX, Bx.T)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:2
3: RuntimeWarning: invalid value encountered in matmul
    hat = Bx @ np.linalg.solve(XtX, Bx.T)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:2
4: RuntimeWarning: divide by zero encountered in matmul
    rss = np.sum((y - Bx @ coef) ** 2)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:2
4: RuntimeWarning: overflow encountered in matmul
    rss = np.sum((y - Bx @ coef) ** 2)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:2
4: RuntimeWarning: invalid value encountered in matmul
    rss = np.sum((y - Bx @ coef) ** 2)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:3

```

```

0: RuntimeWarning: divide by zero encountered in matmul
  XtX = Bx.T @ Bx + lam_opt * (D.T @ D)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:3
0: RuntimeWarning: overflow encountered in matmul
  XtX = Bx.T @ Bx + lam_opt * (D.T @ D)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:3
0: RuntimeWarning: invalid value encountered in matmul
  XtX = Bx.T @ Bx + lam_opt * (D.T @ D)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:3
1: RuntimeWarning: divide by zero encountered in matmul
  coef = np.linalg.solve(XtX, Bx.T @ y)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:3
1: RuntimeWarning: overflow encountered in matmul
  coef = np.linalg.solve(XtX, Bx.T @ y)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:3
1: RuntimeWarning: invalid value encountered in matmul
  coef = np.linalg.solve(XtX, Bx.T @ y)
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:3
9: RuntimeWarning: divide by zero encountered in matmul
  return _basis(t) @ coef
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:3
9: RuntimeWarning: overflow encountered in matmul
  return _basis(t) @ coef
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:3
9: RuntimeWarning: invalid value encountered in matmul
  return _basis(t) @ coef

```



Next, I calculate each symbol's median half-spread during trading hours and take the median of those medians to get my global c .

```

[4]: # Identify best-quote price columns
ask_px_col = 'ask_px_00'

```

```

bid_px_col = 'bid_px_00'

# Half-spread per event
df['half_sp'] = 0.5 * (df[ask_px_col] - df[bid_px_col])

# Keep regular session quotes (09:30-16:00 ET)
session_mask = (
    (df['ts_event_dt'].dt.time >= pd.Timestamp('09:30').time()) &
    (df['ts_event_dt'].dt.time < pd.Timestamp('16:00').time())
)
session_df = df.loc[session_mask]

# Median half-spread per symbol
c_by_sym = (
    session_df.groupby('symbol', as_index=False)['half_sp']
        .median()
        .rename(columns={'half_sp': 'c'})
)

# Global median
c_global = c_by_sym['c'].median()
print(c_global)

```

0.030000000000000001137

Now, we construct an impact curve by averaging, for each order size x , the premium $gt(x)$ paid above the best ask across all snapshots, and then we fit and plot a power-law $g(x) = ax^p$ against those averaged data on log-log axes. We filter out extreme bucket sizes because the data

```

[5]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Build (x, gt) pairs from book snapshots
pairs = []
for _, row in session_df.iterrows():
    base = row['ask_px_00']
    cum_sz = 0.0
    cum_cost = 0.0
    for lvl in range(10):
        sz = row[f'ask_sz_{lvl:02d}']
        px = row[f'ask_px_{lvl:02d}']
        cum_sz += sz
        cum_cost += px * sz
        if lvl == 0:
            continue
        gt = cum_cost / cum_sz - base
        if gt > 0 and cum_sz > 0:

```

```

        pairs.append((cum_sz, gt))

raw = pd.DataFrame(pairs, columns=['x', 'gt'])

# Aggregate to get average gt for each integer size and count samples
raw['x_int'] = raw['x'].astype(int)
avg_df = (
    raw
    .groupby('x_int', as_index=False)
    .agg(gt_avg=('gt', 'mean'), n=('gt', 'count'))
)

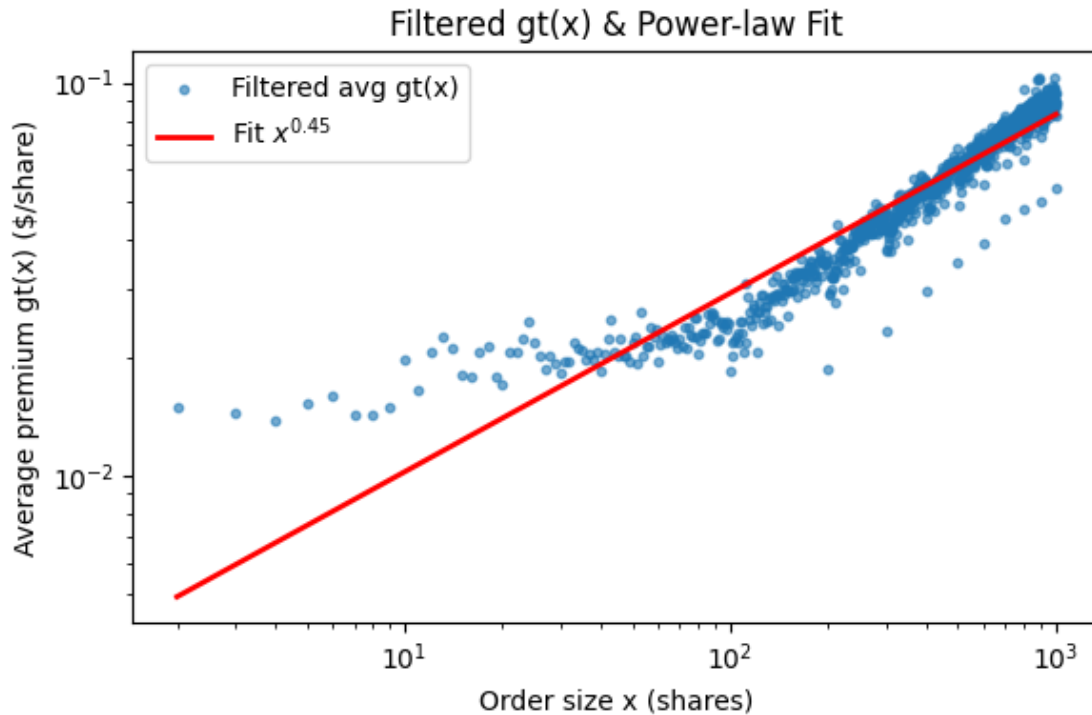
# Filter out under-sampled and extreme-size buckets
filtered = avg_df[avg_df['x_int'] <= 1000]

# Fit power law to the filtered averaged curve
logx = np.log(filtered['x_int'].to_numpy())
logy = np.log(filtered['gt_avg'].to_numpy())
p, loga = np.polyfit(logx, logy, 1)
print(f'p = {p:.3f}')

# Plot filtered average gt(x) and the fitted power-law
plt.figure(figsize=(6, 4))
plt.scatter(filtered['x_int'], filtered['gt_avg'], s=10, alpha=0.6,
            label='Filtered avg gt(x)')
x_fit = np.logspace(np.log10(filtered['x_int'].min()), np.
                    log10(filtered['x_int'].max()), 200)
plt.plot(x_fit, np.exp(loga) * x_fit**p, 'r-', lw=2, label=f'Fit  $x^{{p:.2f}}$ ')
plt.xscale('log')
plt.yscale('log')
plt.xlabel('Order size x (shares)')
plt.ylabel('Average premium gt(x) ($/share)')
plt.title('Filtered gt(x) & Power-law Fit')
plt.legend()
plt.tight_layout()
plt.show()

```

p = 0.455



Finally, we use the mathematical method we decided was best to minimize the $gt(x)$ function, using 390 x_i 's for each minute of the day to represent a chunk of S , the total number of shares we must buy throughout the day. This entails solving for the Lagrange multiplier λ via bisection (using a flat marginal cost c up to depth Dt , and a power-law tail proportional to x^p) and then prints the optimal allocation vector and verifies its sum equals S .

```
[6]: import numpy as np

# Minute grid 0-389
x = np.arange(0, 390)
Dt = Dt_hat(x)
c = c_global
p = 0.45
a = c / Dt**p
T = len(Dt)
S = 10000

total_flat = Dt.sum()
if S <= total_flat:
    # All trades happen in flat region -> proportionally fill Dt
    x_opt = Dt * (S / total_flat)
else:
    # Need tail trades as well -> bisection on
    lam_lo = c
```

```

lam_hi = (p+1) * a.max() * (S**p)
def allocate(lam):
    if lam <= c:
        return Dt.copy()
    xtail = (lam / (a * (p+1)))**(1/p)
    return np.maximum(Dt, xtail)

for _ in range(50):
    lam_mid = 0.5*(lam_lo + lam_hi)
    if allocate(lam_mid).sum() > S:
        lam_hi = lam_mid
    else:
        lam_lo = lam_mid

lam_star = 0.5*(lam_lo + lam_hi)
x_opt = allocate(lam_star)

print("Allocated shares per minute:", x_opt)
print("Check sum =", x_opt.sum())

```

```

Allocated shares per minute: [ 12.97345336  12.9992259   13.02775711
13.05901859  13.09298195
 13.12961877  13.16890067  13.21079923  13.25528607  13.30233278
 13.35191096  13.40399221  13.45854813  13.51555032  13.57497038
 13.63677991  13.70095052  13.76745379  13.83626134  13.90734475
 13.98067564  14.0562256   14.13396623  14.21386913  14.2959059
 14.38004814  14.46626745  14.55453544  14.64482369  14.73710382
 14.83134742  14.92752608  15.02561142  15.12557503  15.22738851
 15.33102346  15.43645149  15.54364418  15.65257315  15.76320998
 15.87552629  15.98949367  16.10508372  16.22226804  16.34101823
 16.46130589  16.58310263  16.70638003  16.83110971  16.95726326
 17.08481227  17.21372836  17.34398313  17.47554816  17.60839506
 17.74249544  17.87782088  18.014343   18.15203339  18.29086365
 18.43080538  18.57181779  18.7138105   18.85668074  19.00032574
 19.14464274  19.28952896  19.43488163  19.58059799  19.72657526
 19.87271067  20.01890146  20.16504486  20.31103809  20.45677839
 20.60216298  20.7470891   20.89145398  21.03515484  21.17808892
 21.32015345  21.46124566  21.60126278  21.74010204  21.87766066
 22.01383589  22.14852495  22.28162506  22.41303347  22.5426474
 22.67036408  22.79608075  22.91969462  23.04110294  23.16020293
 23.27689183  23.39106686  23.50262525  23.61146424  23.71748105
 23.82057292  23.92063707  24.01757074  24.11127116  24.20163555
 24.28856115  24.37194519  24.45168489  24.5276775   24.59982023
 24.66801032  24.732145   24.7921215   24.84783705  24.89918888
 24.94611316  24.98870187  25.0270859   25.06139616  25.09176357
 25.11831902  25.14119343  25.1605177   25.17642275  25.18903947
 25.19849878  25.20493158  25.20846878  25.20924129  25.20738002
 25.20301587  25.19627975  25.18730257  25.17621524  25.16314866

```

25.14823374	25.13160139	25.11338251	25.09370802	25.07270883
25.05051583	25.02725993	25.00307205	24.9780831	24.95242397
24.92622558	24.89961883	24.87273463	24.84570389	24.81865752
24.79172643	24.76504151	24.73873369	24.71293386	24.68777293
24.66338182	24.63989143	24.61743266	24.59613643	24.57613364
24.55755552	24.54053202	24.525195	24.51167505	24.50010309
24.49061001	24.48332672	24.47838414	24.47591316	24.47599765
24.47853324	24.48336852	24.49035206	24.49933246	24.51015829
24.52267814	24.5367406	24.55219424	24.56888765	24.58666942
24.60538812	24.62489235	24.64503068	24.6656517	24.68660399
24.70773614	24.72889674	24.74993435	24.77069757	24.79103499
24.81079518	24.82982673	24.84797822	24.86509823	24.88103536
24.89563818	24.90875528	24.92023524	24.92992665	24.93767808
24.94333813	24.94675537	24.94777839	24.94625577	24.94203611
24.93496797	24.92489995	24.91168062	24.89515858	24.8751824
24.85160068	24.82426198	24.79301491	24.75770803	24.71818994
24.67430922	24.62591444	24.57285421	24.51497709	24.45213167
24.38416655	24.31093029	24.23227148	24.14812676	24.05878487
23.96462262	23.86601682	23.76334427	23.65698177	23.54730612
23.43469413	23.3195226	23.20216833	23.08300813	22.9624188
22.84077713	22.71845994	22.59584402	22.47330618	22.35122323
22.22997195	22.10992917	21.99147167	21.87497626	21.76081975
21.64937894	21.54103063	21.43615162	21.33511871	21.23830872
21.14609843	21.05886466	20.9769842	20.90083387	20.83079045
20.76723076	20.7105316	20.66106977	20.61922207	20.5853653
20.55987627	20.54313179	20.53550865	20.53738365	20.5491336
20.57113531	20.60376557	20.64740118	20.70241896	20.7691957
20.8481082	20.93953327	21.04384771	21.16142833	21.29265192
21.43789529	21.59753524	21.77174455	21.95987989	22.16109391
22.37453925	22.59936857	22.8347345	23.07978969	23.33368679
23.59557845	23.86461732	24.13995603	24.42074724	24.70614359
24.99529773	25.28736231	25.58148997	25.87683337	26.17254513
26.46777792	26.76168438	27.05341715	27.34212889	27.62697223
27.90709983	28.18166433	28.44981838	28.71071463	28.96350571
29.20734429	29.441383	29.66477449	29.8766714	30.0762264
30.26259211	30.43492119	30.59236628	30.73408004	30.8592151
30.96692412	31.05635973	31.1266746	31.17702135	31.20655265
31.21442113	31.19977945	31.16178025	31.09957617	31.01231986
30.89916398	30.75926116	30.59176405	30.39582531	30.17059756
29.91523347	29.62976961	29.31777824	28.98371555	28.63203776
28.26720104	27.8936616	27.51587564	27.13829933	26.7653889
26.40160052	26.05139039	25.71921472	25.40952969	25.1267915
24.87545634	24.65998042	24.48481992	24.35443105	24.27326999
24.24579295	24.27645611	24.36971568	24.53002785	24.76184882
25.06963477	25.45784192	25.93092644	26.49334454	27.14955242
27.90400626	28.76116227	29.72547664	30.80140556	31.99340523
33.30593185	34.74344161	36.31039071	38.01123533	39.85043169
41.83243597	43.96170437	46.24269308	48.67985831	51.27765624

```

54.04054307 56.97297499 60.07940821 63.36429892 66.83210331
70.48727758 74.33427792 78.37756053 82.6215816 87.07079734
91.72966393 96.60263758 101.69417447 107.0087308 112.55076277]
Check sum = 10000.000000000002

/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:3
9: RuntimeWarning: divide by zero encountered in matmul
    return _basis(t) @ coef
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:3
9: RuntimeWarning: overflow encountered in matmul
    return _basis(t) @ coef
/var/folders/8_/l7ym394s7wzd2wrk9f9333qh0000gn/T/ipykernel_15550/2757330266.py:3
9: RuntimeWarning: invalid value encountered in matmul
    return _basis(t) @ coef

```